

DOCUMENTACIÓN DEL PORTAL.

Cómo está construido el portal post-purchase de LIT,
qué hace cada superficie, y las decisiones técnicas
detrás del producto.

00 ÍNDICE

01	RESUMEN EJECUTIVO	P. 3
02	STACK Y ARQUITECTURA	P. 4
03	AUTENTICACIÓN Y SESIONES	P. 6
04	SUPERFICIES DEL PORTAL	P. 8
05	INTEGRACIONES	P. 14
06	DECISIONES DIFÍCILES Y APRENDIZAJES	P. 16
07	BACKLOG Y PRÓXIMOS PASOS	P. 18

01 RESUMEN EJECUTIVO

El **Portal de Cliente de LIT** es la nueva área privada donde el suscriptor gestiona su caja: cambia el plan, salta envíos, modifica la dirección, añade extras a la próxima caja, y, llegado el caso, cancela. Sustituye al portal genérico de *tracking* que venía por defecto con la herramienta de suscripciones (Seal).

EL POR QUÉ

El portal antiguo era una página técnica, fea, con muchos enlaces, y vivía en un dominio distinto (`tracking.litsalt.com`) — rompía la marca. Construir uno propio significa que el cliente **jamás sale de litsalt.com**, ve la identidad LIT en todo momento, y nosotros controlamos cada flujo (cancelaciones, win-back, recompensas) en lugar de depender de lo que Seal nos ofrezca.

¿QUÉ CONSIGUE EL CLIENTE?

- **Ver su próxima caja** con la fecha exacta, el sabor, y cuántos sobres recibirá.
- **Saltar una entrega** en un click si se va de viaje, sin tener que escribir a soporte.
- **Cambiar el plan** (cajas + frecuencia) con precio dinámico — ve el descuento por volumen en vivo.
- **Modificar dirección y método de pago** sin tener que llamar a nadie.
- **Cancelar** con un flujo en 4 pasos que muestra alternativas (saltar, cambiar plan) antes de la decisión final.
- **Ver historial** de pedidos, calendario de próximas entregas, y, en fase 2, su colección de cartas y recompensas Drops.

¿QUÉ CONSIGUE LIT?

- **Reducción de tickets a soporte** — la mayoría de "skip mi caja" / "cambiar plan" pasan a self-service.
- **Marca consistente** de extremo a extremo. Misma tipografía, mismo idioma, mismo trato.
- **Recuperación de cancelaciones** integrada en el propio flujo de cancel — antes el cliente cancelaba sin que LIT tuviera oportunidad de ofrecer alternativas.
- **Telemetría limpia**: cada acción se loguea en Klaviyo (cancellation reason, skip, plan change) para alimentar flujos automáticos de retención.

ESTADO ACTUAL

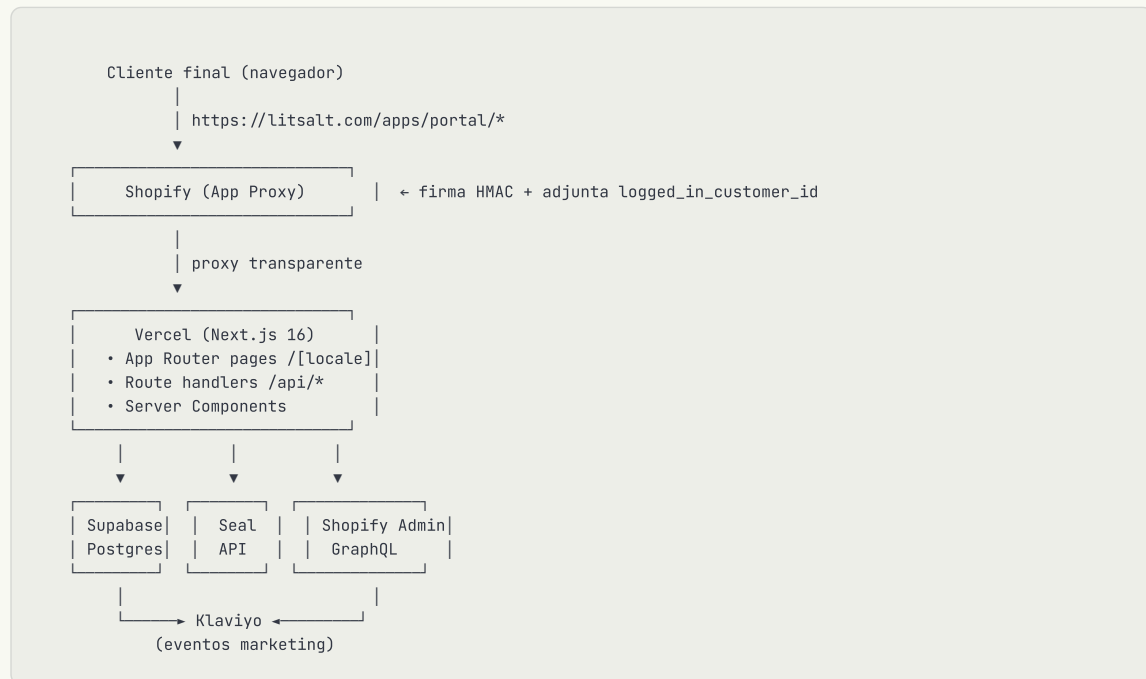
El portal está **desplegado en producción** en `litsalt.com/apps/portal/*`. Los flujos críticos (login OAuth, ver suscripción, saltar, cambiar plan, cambiar dirección, cancelar) están funcionando end-to-end. Pendiente principal: **Fase 2** con Collection (cartas), Drops automation (recompensas) y el switcher de sabores cuando haya catálogo.

02 STACK Y ARQUITECTURA

2.1 EL STACK EN UNA FRASE

Next.js 16 (App Router) alojado en **Vercel**, servido al cliente vía **Shopify App Proxy**, con **Supabase Postgres** como base de datos, e integrado contra **Seal Subscriptions**, **Shopify Admin API** y **Klaviyo**.

2.2 DIAGRAMA DE ALTO NIVEL



2.3 POR QUÉ ESTE STACK

PIEZA	DECISIÓN	POR QUÉ
Next.js + App Router	vs SPA pura	Server components para precargar datos sensibles sin exponer tokens al bundle. SSR para SEO mínimo y first-paint rápido.
Shopify App Proxy	vs subdominio propio	El cliente nunca sale de litsalt.com – no hay corte de marca ni segundo login. Shopify firma cada request con HMAC y nos adjunta el <code>logged_in_customer_id</code> .
Vercel	vs Hetzner/VPS	Despliegue continuo desde GitHub, edge functions, zero-config. El portal escala con tráfico de mañana sin tocar infra.
Supabase	vs Neon (que usamos para LIT-Agents)	Postgres gestionado + Auth + tipos en tiempo real. Nos da una DB sin operarla y un panel para ops review de cancellations.
Seal Subscriptions	—	Es el motor de suscripciones que ya teníamos antes del portal. El portal habla con su Merchant API.
Customer Account API	vs Classic Accounts	OAuth moderno con PKCE; sustituye el login viejo. Soporta el flujo de "venir desde un email" sin re-logout.

2.4 ESTRUCTURA DEL REPO

```
portal-cliente-web/lit-portal/
├─ src/app/
├─ [locale]/
└─ # páginas /es/* y /en/*
```

```
| | | | | mi-lit/           # Hub (home del portal)
| | | | | cuenta/        # Account
| | | | | collection/     # Cartas (fase 2)
| | | | | the-world/      # Eventos/historias (fase 2)
| | | | | auth/handoff/    # Transferencia de session_id
| | | | | └─ api/         # Route handlers (mutations + reads)
| | | | |   | auth/       # OAuth login/callback/logout
| | | | |   | subscription/ # Plan, skip, cancel, extras, address
| | | | |   | hub/dashboard # Resumen agregado del Hub
| | | | |   └─ webhooks/   # Seal + Shopify webhooks
| | | | | └─ src/components/ # Overlays (PlanOverlay, SkipOverlay...), Hero, etc.
| | | | | └─ src/lib/      # Clients: seal, shopify-admin, klaviyo, supabase, cutoff, just-skipped
| | | | | └─ database/
| | | | |   | schema.sql   # Schema Postgres canónico
| | | | |   └─ migrations/ # Diff migrations aplicadas en prod
| | | | | └─ scripts/      # Upload templates Klaviyo, debug, etc.
| | | | | └─ docs/         # Este documento + spec + auditoria
```

03 AUTENTICACIÓN Y SESIONES

El portal soporta **dos mecanismos de autenticación** simultáneos, que conviven en el middleware `withCustomer`. Cualquiera de los dos vale para identificar al cliente. Esto es así porque la mayoría de los clientes entrarán vía un enlace en un email de Shopify (App Proxy auth), pero también queremos permitir login directo desde la web (OAuth).

3.1 VÍA A: APP PROXY (PASSIVE)

Cuando Shopify reenvía una request a Vercel, añade `logged_in_customer_id` como parámetro de query y firma toda la URL con HMAC usando el secret de la app. Nuestro middleware verifica esa firma y, si es válida, identifica al cliente sin más pasos.

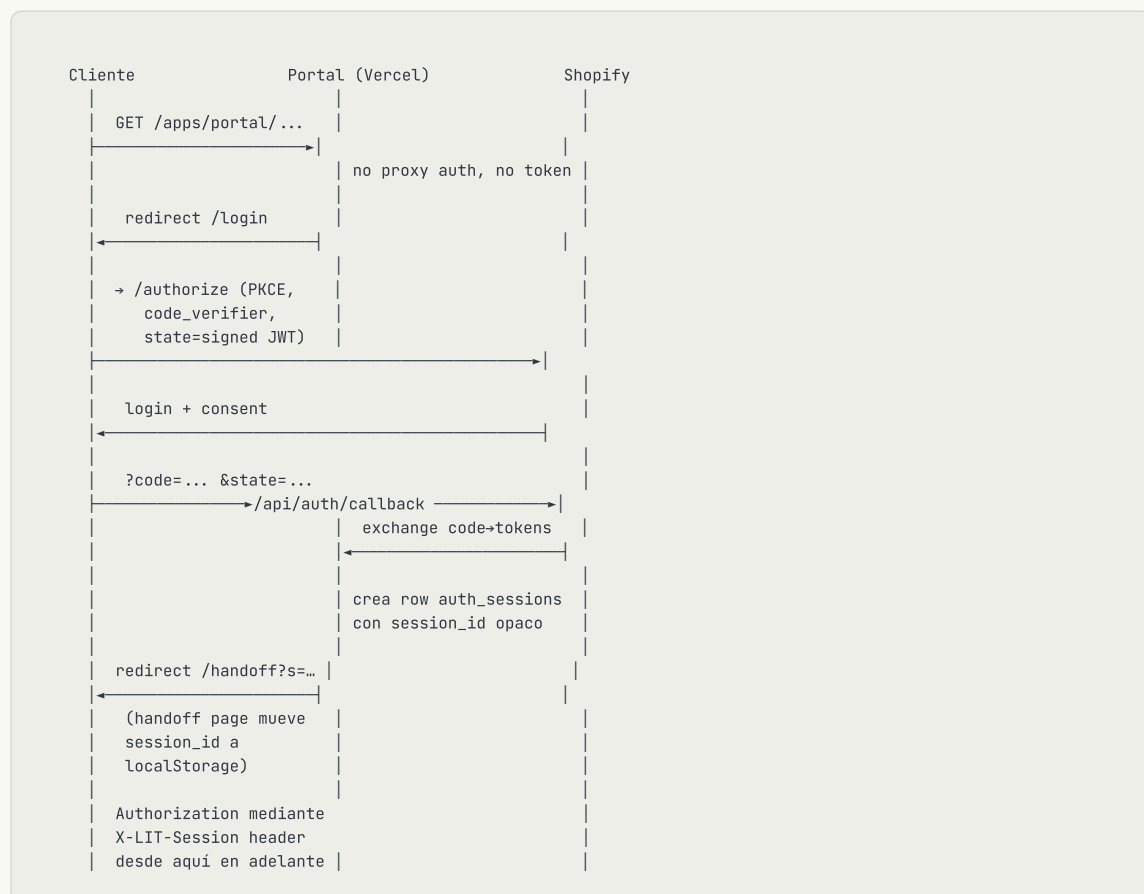
VENTAJA

Es transparente para el cliente. Si está logueado en la storefront de Shopify (cookie de Shopify), cuando hace click en cualquier enlace al portal entra ya identificado, sin pasar por una pantalla de login.

3.2 VÍA B: OAUTH CON CUSTOMER ACCOUNT API (ACTIVE)

Cuando no hay sesión de Shopify (entró directo a la URL del portal sin venir de Shopify), redirigimos al cliente al login de Shopify (Customer Account API), hacemos un flujo OAuth con PKCE, y al volver del callback creamos un **session_id opaco** en nuestra tabla `auth_sessions` en Supabase. El FE guarda ese session_id en localStorage y lo manda en cada request en el header `X-LIT-Session`.

3.3 DIAGRAMA DEL OAUTH FLOW



3.4 POR QUÉ X-LIT-SESSION Y NO AUTHORIZATION: BEARER

GOTCHA DESCUBIERTO EL 2026-05-21

Shopify App Proxy **intercepta el header** `Authorization` en peticiones POST/PATCH/DELETE y NO las reenvía al upstream. En su lugar, devuelve un 500 con el HTML del tema (la home de la tienda), lo cual hace que el FE crea que el endpoint está roto. Headers custom `X-*` pasan sin tocarlos. Migramos a `X-LIT-Session` y todas las mutations volvieron a funcionar.

3.5 LOGOUT

El endpoint `/api/auth/logout` borra la fila de `auth_sessions` en Supabase, devuelve la URL de logout OIDC de Shopify para invalidar también la sesión storefront, y el FE limpia su `localStorage` antes de redirigir.

04 SUPERFICIES DEL PORTAL

El portal está dividido en superficies, cada una con una URL y una responsabilidad clara. Todas comparten el mismo header móvil fijo, las mismas tipografías (Archivo Black para títulos, Barlow para body) y los mismos colores brand (cream #F8F9F2, lit-grey #323743, bold-yellow #EBEE62).

4.1 MI LIT (HUB)

/apps/portal/[es|en]/mi-lit

HOME

POST-LOGIN

ruta primaria

Pantalla principal del cliente. Lo primero que ve después de loguearse. Estructura:

- **Hero "Next Box"** — fecha grande del próximo envío, sabor, plan. Tiene 4 variantes según el estado: `default`, `skipped` (saltaste el anterior, este es el próximo), `locked` (dentro de 24h del envío, no se puede tocar), `new` (es tu primera caja).
- **Quick Actions** — 4 botones rápidos: Cambiar plan, Saltar próxima, Cambiar sabor (Fase 2), Extras (Fase 2). Cada uno abre su overlay correspondiente.
- **Calendar de próximas entregas** — muestra las próximas 5-6 fechas que Seal tiene pre-agendadas.
- **Mini Collection grid** — preview de la colección de cartas (Fase 2).
- **Order history** — últimos 10 pedidos completados, con su tracking si está disponible.

4.2 CUENTA

/apps/portal/[es|en]/cuenta

ACCOUNT

SETTINGS

Misma información que Mi LIT pero formateada como ficha: datos personales (nombre, email, teléfono), suscripción actual (cajas, frecuencia, sabor, próxima fecha), dirección, método de pago, historial completo de pedidos. Incluye la sección "Cambiar cajas o frecuencia" como acceso rápido al plan overlay y, al final, la **Danger Zone** con "Cerrar sesión" y "Cancelar suscripción".

Tras el último deploy, los Quick Actions de Cuenta funcionan exactamente igual que los de Mi LIT (mismas reglas de disabled, mismo banner amarillo "Saltaste la entrega anterior — Deshacer" cuando hay un skip activo).

4.3 SKIP OVERLAY

OVERLAY

MUTATION

Bottom-sheet en móvil, modal centrado en desktop. Muestra "¿Te vas de viaje?", la fecha de envío actual, y un único botón "Confirmar saltar". Al confirmar:

1. POST `/api/subscription/skip` con `sealSubscriptionId` en el body (fast-path).
2. Backend valida ownership en Supabase, hace un GET dirigido a Seal por ID (sin pagar), comprueba ventana de cutoff (24h), llama `skipBillingAttempt`.
3. FE recibe la nueva fecha, marca `justSkipped` en localStorage con expiry hasta esa fecha, dispara evento Klaviyo.

4.4 PLAN OVERLAY

OVERLAY

MUTATION

Selector de cajas (1 a 6) + selector de frecuencia (15d hasta 6 meses). Muestra precio dinámico desde Shopify, con `compareAtPrice` tachado si aplica descuento. Si hay un skip activo, muestra banner amarillo: *"Al cambiar el plan, tu salto de caja se deshará"*. Al guardar:

1. Si cambia la **frecuencia**: `seal.editSubscription(delivery_interval)` primero.
2. Si cambia el **nº de cajas**: `seal.addItems` con la nueva variant, luego `removeItems` de la antigua. Seal no permite swap directo.
3. Pausas de 500ms entre mutations porque Seal regenera `billing_attempts` en background y la siguiente mutation puede ser silenciosamente ignorada si llega demasiado pronto.
4. Verificación post-mutation con `AbortController` de 4s. Si timed out, devuelve respuesta sintética y el FE arranca un re-poll silencioso de 60s.

4.5 CANCEL TAKEOVER (FULL-SCREEN)

FULL SCREEN

MULTI-STEP

WIN-BACK

No es un overlay, es un takeover oscuro a pantalla completa que ocupa toda la viewport. 4 pasos:

1. **Stats** — "Has recibido N cajas y llevas M meses en LIT". Reconocer el viaje del cliente antes de proponer salirse.
2. **Alternativas** — botones para Saltar próxima caja, Cambiar plan, o "Nuevos sabores próximamente" (disabled). Pivotan al overlay correspondiente sin cerrar el takeover.
3. **Motivo** — selector de razón (caro / demasiado producto / no lo uso suficiente / pausa / otro) + free text opcional.
4. **Confirmación** — muestra "Cancelación inmediata" o "Cancela tras último envío" según si está dentro de 24h del próximo envío. Botón final "Cancelar suscripción".

Al confirmar: marca cancellation row en Supabase como `committing`, llama Seal `cancelSubscription`, promueve a `confirmed`, dispara Klaviyo event, y muestra el done state. El botón "Volver a LIT" del done state hace **logout completo** (borra sesión, redirige a `litsalt.com`) — el cliente no vuelve al portal con estado stale.

4.6 LOGIN / EMPTYSTATE

/apps/portal/[es|en]/mi-lit (sin auth)

FALLBACK

Si el cliente entra sin sesión, ve la **LoginScreen** con CTA a OAuth. Si entra autenticado pero no tiene una suscripción activa (one-shot buyer, cancelado), ve el **EmptyState**: "Gracias por probar LIT" + CTA a la página de suscripción con descuento "desde el 25%".

4.7 ADDRESS OVERLAY

OVERLAY

MUTATION

Formulario para cambiar la dirección de envío. PATCH a `/api/subscription/address`, que llama a `seal.updateShippingAddress`. Aplica cutoff de 24h: si el próximo envío está demasiado cerca, no se permite el cambio.

4.8 PAYMENT METHOD

EMAIL-ONLY

Decisión locked 2026-05-19: el cambio de método de pago es **100% email-only**. Botón "Cambiar método de pago" dispara `customerPaymentMethodSendUpdateEmail` de Shopify Admin API, que envía un enlace seguro al email del cliente. El portal muestra confirmación "Te hemos enviado un enlace a tu correo, vuelve aquí al terminar". **Sin Stripe, sin pasarela nueva** — Shopify ya tiene el método y la pasarela.

05 INTEGRACIONES

5.1 SEAL SUBSCRIPTIONS

El portal lee y muta el estado de las suscripciones únicamente a través de la **Merchant API de Seal**. Endpoints usados:

ACCIÓN	ENDPOINT SEAL	NOTAS
Listar subs por email	GET <code>/subscriptions?page=N</code>	Sin filtro server-side, hay que paginar. Hasta 33 páginas para cuentas grandes.
Sub por ID	GET <code>/subscription?id=X</code>	Endpoint singular que si honra el filtro, evita la paginación. Descubierta 2026-05-21.
Cambiar intervalo	PUT <code>/subscription</code> <code>action=edit</code>	Solo <code>delivery_interval</code> , NO mandar <code>billing_interval</code> (Seal silenciosamente no aplica el cambio).
Cambiar variant	<code>add_items</code> + <code>remove_items</code>	No hay endpoint de swap. Hay que añadir nuevo + quitar viejo.
Saltar billing attempt	PUT <code>/subscription-billing-attempt</code> <code>action=skip</code>	Retornar 200 con <code>success: false</code> es posible — hay que chequear el body.
Cancelar / reactivar	PUT <code>/subscription</code> <code>action=cancel reactivate</code>	Cancel es INMEDIATO (Seal pone <code>cancelled_on</code> en el momento).

5.2 SHOPIFY ADMIN API

Usada para datos del cliente y operaciones que Seal no expone:

- **Lookup de email** por `customer_id` (App Proxy nos da el ID, pero Seal indexa por email).
- **Actualizar customer** (nombre, teléfono, language pref).
- **Catalog de extras** — productos con tag `add-to-box` que se pueden añadir a la siguiente caja.
- **Send payment method update email** via `customerPaymentMethodSendUpdateEmail`.
- **Order history** — últimos pedidos del cliente para el historial.

5.3 KLAVIYO

Cada acción importante dispara un evento custom en Klaviyo, que alimenta flujos de marketing y win-back automatizados. Eventos actuales:

- `subscription_skip`
- `subscription_cancelled` (con reason, freeText, cancelCount, lastShipDate, dropsHeldUntil)
- `welcome_to_portal` (al crear sesión OAuth por primera vez)

Las plantillas de email viven en Klaviyo y se sincronizan con el repo mediante `scripts/klaviyo-upload-templates.mjs`. Hay dos plantillas welcome (ES `VXC2NN`, EN `SCE2Q2`) con CTA al portal.

5.4 SUPABASE

Tablas principales:

TABLA	PROPÓSITO
<code>auth_sessions</code>	<code>session_id</code> ↔ <code>customer_id</code> , expiry, id_token para logout OIDC.
<code>subscriptions</code>	Cache de mapping <code>customer_id</code> ↔ <code>seal_subscription_id</code> . Evita el scan de 33 páginas en cada request.

TABLA	PROPÓSITO
cancellations	Multi-step cancel: pending → committing → confirmed. Permite detectar cancels colgados que no terminaron de aplicarse.
customer_preferences	cancel_count, language_pref, last_cancelled_at — para lógica de "second cancel = reset Drops inmediato".
drops_balances / drops_events	Saldo de Drops y log de cada evento (Fase 2).
webhook_log	Idempotency log de webhooks Shopify y Seal.

5.5 SHOPIFY APP PROXY

Configurado en `shopify.app.toml` :

```
[app_proxy]
url      = "https://lit-portal-drab.vercel.app"
subpath  = "portal"
prefix   = "apps"
```

Cualquier request a `litsalt.com/apps/portal/*` se reenvía con HMAC + `logged_in_customer_id` + `shop` + `timestamp` . El middleware `verifyAppProxyRequest` valida la firma antes de procesar.

06 DECISIONES DIFÍCILES Y APRENDIZAJES

Esta sección documenta las decisiones no obvias y los gotchas que aprendimos por las malas, para que no se repitan ni se olviden.

6.1 SHOPIFY APP PROXY SUSTITUYE 5XX CON HTML DEL STOREFRONT

EL GOTCHA

Cuando el upstream (Vercel) devuelve un HTTP 5xx, App Proxy **NO reenvía esa respuesta al cliente**. En su lugar, renderiza el HTML del tema de la tienda con la URL solicitada como canonical. El cliente recibe ~50KB de markup del storefront con status 500, pero `content-type: text/html` y headers de Shopify (`powered-by: Shopify` , `server-timing: theme;desc=...`).

Esto hizo que un error real (constraint CHECK violado en la tabla `cancellations`) pareciera un problema de routing, de Authorization header, de timeout — todo equivocado. **Diagnóstico:** hacer bypass del proxy y hablar directo con Vercel (`curl https://lit-portal-drab.vercel.app/...`) para ver el error JSON real.

6.2 SEAL REGENERA `BILLING_ATTEMPTS` EN CADA CAMBIO DE PLAN

Cualquier `editSubscription` en Seal (cambiar cajas, frecuencia, dirección, items) hace que regenere la lista de `billing_attempts` desde cero. **Cualquier skip aplicado previamente se borra silenciosamente**. El portal mitiga esto:

- El FE limpia el flag `justSkipped` de localStorage cuando hay un plan change exitoso, para no mostrar un banner que miente.
- El PlanOverlay avisa al usuario con un banner amarillo si hay un skip activo y va a cambiar el plan.

6.3 EVENTUAL CONSISTENCY EN SEAL

Después de una mutation, un GET inmediato puede devolver datos stale (el attempt aparece como pendiente cuando ya está marcado skipped). Solución en el código:

- El skip endpoint **computa el nuevo nextShipDate localmente** en lugar de re-fetchear, porque sabe qué attempt se marcó.
- El plan endpoint tiene un re-poll con AbortController y timeout de 4s.
- El FE arranca un re-poll silencioso de 60s tras un plan change.

6.4 CARÁCTER "—" (EM-DASH) PROHIBIDO EN TODO EL COPY

Hard rule de marca: ningún copy de LIT (portal + emails + ads) puede llevar un em-dash. Usar coma o punto. Esto es una preferencia del founder y se ha mantenido consistente.

6.5 LA VENTANA DE CUTOFF ES 24H, ANTES ERA 72H

Decidido el 2026-05-21: el operador interno necesita al menos 24h para procesar cambios antes del envío. Antes eran 72h, que bloqueaba demasiados cambios que sí se podían atender. Se aplica a skip, plan, address, extras.

6.6 CANCEL ES IMMEDIATE EN SEAL

El comportamiento por defecto de Seal cancel pone `canceled_on` al instante, no después del último envío. El portal lo refleja en el copy de confirmación: *"Cancelación inmediata. No habrá más envíos ni cobros"*, excepto si el próximo envío está dentro de 24h y ya se ha procesado, en cuyo caso ese último sí sale.

6.7 SCHEMA MIGRATION APLICADA EN PROD

La constraint `cancellations_status_check` originalmente solo permitía `pending` y `confirmed` . Se añadió `committing` como estado intermedio (entre que se llama a Seal y se confirma el éxito) — pero la migration nunca se

ejecutó hasta el 2026-05-21. Como resultado, durante semanas **nadie podía cancelar**. Ahora la migration vive en `database/migrations/` y el schema canónico se mantiene sincronizado.

6.8 EL CLIENTE JAMÁS VE `TRACKING.LITSALT.COM`

Regla dura del portal: el cliente final no puede ver, en ningún momento, el portal de tracking antiguo. Todo el flujo de su cuenta vive dentro de `litsalt.com/apps/portal/*`. Si hay que cambiar método de pago, se hace por email. Si hay que cancelar, se hace en el takeover. Si hay un error, se muestra inline. El portal viejo "dejará de existir" para el cliente final.

07 BACKLOG Y PRÓXIMOS PASOS

7.1 FASE 2 (POST-LAUNCH)

FEATURE	ESTADO	NOTAS
Collection (cartas)	Stub UI	Mini grid en Hub renderiza placeholders. Falta backend (drops_events, card unlocks).
Drops automation	Schema OK	Tablas listas (drops_balances , drops_events). Falta: trigger por box shipped, referral, monthly streak, etc.
Inner Circle (tier reward)	Stub	TierPill aparece si tier_earned . Falta evento de earn + reward unlock.
Flavor switcher	Disabled	QA muestra "Próximamente". Activar cuando haya catálogo con >1 sabor.
Extras catalog	API lista	Backend completo (POST /api/subscription/extras). UI marca como "Próximamente" hasta que Shopify tenga productos con tag add-to-box .
The World (eventos)	Schema	Tablas events/moments/stories listas. Falta admin tool para crear contenido.
Barcelona waitlist	API lista	Endpoint funciona, falta UI en The World.

7.2 HARDENING PENDIENTE (DE LA AUDITORÍA)

La auditoría de seguridad del 2026-05-21 (documento separado en docs/SECURITY_AUDIT_2026-05-21.md) identificó 13 hallazgos críticos/altos. Los 3 más prioritarios:

1. **Stop the bleeding**: quitar logs de tokens, dejar de devolver stack traces al cliente, mover session_id del query string al URL fragment en el handoff.
2. **Apretar ownership end-to-end**: verifyOwnershipFast debe filtrar también por seal_subscription_id ; validar mainItemId ∈ sub.items antes de remove; bloquear cambio de email desde el portal.
3. **Reparar la auth crypto**: verificar id_token vía Shopify JWKS, validar timestamp de App Proxy (±60s), derivar keys separadas para HMAC y JWT state, fix logout para leer X-LIT-Session.

7.3 MEJORAS DE DX Y OPS

- **Rate limiting** en mutations críticas (token bucket por customer).
- **Dashboard ops** para ver cancellations stuck en committing y resolverlas manualmente.
- **Sentry o equivalent** para tracking de errores en producción (hoy solo hay console.log a Vercel logs).
- **E2E tests con Playwright** para los flujos críticos (skip, plan change, cancel).
- **Webhook log replay** tool para reprocesar eventos Shopify/Seal que fallaron.

7.4 TAREAS CONOCIDAS QUE EL FOUNDER HARÁ MANUALMENTE

- Cambiar el link del icono del header en el storefront de /account a /apps/portal/mi-lit (Juan lo hace en Shopify Liquid editor).
- Cambiar el CTA del email de Order Confirmation de Shopify para que apunte al portal en vez de a tracking.litsalt.com (Vía B "stand-by").
- Configurar el filtro de Klaviyo flow del welcome email para que solo dispare con pedidos de suscripción, no one-shot.

EN RESUMEN

El portal está vivo, los flujos críticos funcionan end-to-end, y el cliente puede gestionar su suscripción sin tocar el portal antiguo. Queda hardening de seguridad (priorizado en el doc anexo), Fase 2 de Collection/Drops/Inner

Circle (schema listo, falta UI + triggers) y unas tareas operacionales del founder. La base es sólida para lanzar a clientes reales después del hardening.

Documento generado el 2026-05-21. Fuente: branch `feat/master-spec-rewrite` commit `97837d6` . Para detalles de seguridad ver `docs/SECURITY_AUDIT_2026-05-21.md` . Para spec original ver `portal-cliente-web/lit-portal/MASTER_SPEC.md` (si existe en el repo).